

Created by: Nicolas Chauvaux (n.chauvaux@tudelft.nl), Douwe den Blanken (d.m.j.denblanken@tudelft.nl) and Guilherme Guedes (g.guedes@tudelft.nl), Jan 2026

1 Introduction

In this laboratory exercise, you will perform synthesis and post-synthesis simulation on a baseline Cooley-Tukey FFT algorithm using the same scripts as you used at the end of lab 2. This way, you will have a starting design that you can modify (or change completely) for your project, hopefully aiding you in the accelerator design setup.

Note that you **do not** have to submit a report about this extra lab: this lab is purely to get you familiar with the implementation of an accelerator that can be communicated with from the PicoSoC. This will also provide an example of properly implementing a finite state machine (FSM) in Verilog.

Finally, if you have questions about the labs and project, please post them on Brightspace's Q&A forum.

2 Baseline accelerator design

The provided baseline accelerator computes the Fourier transform of an audio sample. To achieve this, it follows the Cooley-Tukey FFT algorithm¹ for which the software baseline was provided to you in lab 1.

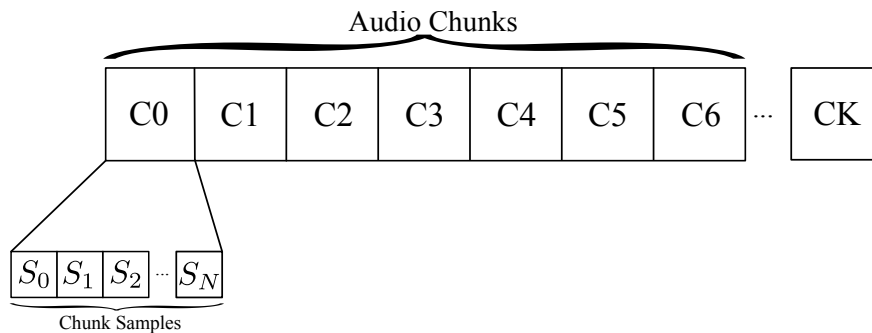


Figure 1: Difference between Chunks and Samples. An audio signal is first divided into chunks with the same duration and each chunk is processed independently, being sampled at 32Hz.

In/out To function, the accelerator needs access to the sampled audio chunks. The accelerator contains a register array of 32 by 32 bits for this. **The RISC-V must first write the data into the accelerator register file before enabling the accelerator.** The implementation in

¹<https://vanhunteradams.com/FFT/FFT.html#The-Cooley-Tukey-FFT>

Verilog of this accelerator memory can be found in the files

`/data/labs/2026/labs/lab3_preparation/src/design/accelerator_mem.v`. The number of samples and number of FFT stages are stored in the `number_data` and `fft_stages`. Each sample of an audio chunk (Fig.1) is processed individually. There are two types of test you will be running, one that runs a **single audio chunk**, where 32 samples are processed, and a larger test where **several chunks** of 32 samples will be processed. The latter will be used as your benchmarking test, whereas the first should be used to help you test ideas and debug your algorithm

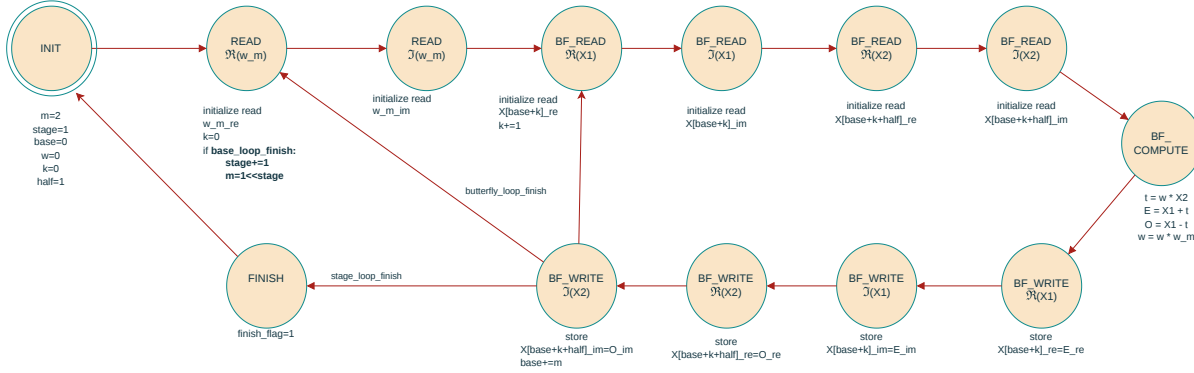


Figure 2: Moore machine state diagram of the baseline FFT accelerator.

Implementation The proposed accelerator is designed using a Moore machine, a type of finite state machine (FSM). Another variant of FSM is the Mealy machine. Check out this website to understand the difference between the two. Properly understanding those differences may help you in your project. As figure 2 shows, the accelerator implements the FFT algorithm using a finite-state machine with 13 states:

- **IDLE:** Initial state where the accelerator waits for the start signal and initializes data structures when triggered. The base value is set to 0 while the stage and twiddle values are set to 1.
- **READ_X States:** States in which the Real or Imaginary parts of the Twiddles (w), or the samples are read from the accelerator memory.
- **BF_COMPUTE:** Main computation state, where the current butterfly elements are processed.
- **WRITE_X:** States where the in place write operation takes place, replacing the old value by its resulting FFT value.
- **FINISH:** All the butterfly elements have been processed, set a finish flag and send the accelerator into idle state.

The implementation in Verilog of this accelerator can be found in the files

`/data/labs/2026/labs/lab3_preparation/task/src/design/accelerator.v` and

`/data/labs/2026/labs/lab3_preparation/task/src/design/accelerator_fft.v`

whereas the related C-code is located at `/data/labs/2026/labs/lab3_preparation/task/firmware/accel.c`.

Have a look at the files, understand them, and try to link the code to the state diagram in figure 2.

3 Task 1: Synthesize PicoSoC

In this task, you will synthesize the whole `PicoSoC` with the Cooley-Tukey's FFT accelerator that will be used as a baseline for the project. To start, run the behavioral simulation of the full system, the RISC-V core, and the accelerator. Copy the code and source the setup script via:

```
1 cd ~
2 cp -r /data/labs/2026/labs/lab3_preparation ~/
3 cd lab3_preparation
4 source setup.sh
5 cd task
```

Then compile the C software and run the behavioral python simulation via:

```
1 cd firmware && make # Compile the C-interface with the accelerator
2 cd ../sim_behav/ && source run_behav_sim.sh # Actually run the hardware-software
  simulation
```

Once finished, you can either visually inspect that the FFT coefficients have been correctly computed or you can run:

```
1 python verify.py
```

If everything looks good, then you can run the synthesis of the full accelerator from a **Bash** shell terminal (this runs the exact same scripts as in lab 2) via:

```
1 cd ../synth
2 source run_synth.sh
3 exit
```

4 Task 2: Post-Synthesis Simulation of PicoSoC

Navigate to the `sim_struct` folder:

```
1 cd ../sim_struct
```

Run the post-synthesis simulation with:

```
1 source run_struct_sim.sh
```

```
1 python verify.py
```

You are successful if `Test Passed ^_^ Outputs and Gold are identical!!!` is printed in your terminal.